1. Interface Servo Motor with Single Cycle

```
import RPi.GPIO as g        # Import Raspberry Pi GPIO library
import time                 # Import time library for delay
g.setmode(g.BOARD)          # Use board pin numbering
g.setwarnings(False)        # Disable GPIO warnings
g.setup(11, g.OUT)          # Set pin 11 as output (connected to servo signal pin)
servo = g.PWM(11, 50)       # Create PWM on pin 11 with frequency 50Hz
servo.start(5)              # Start PWM with duty cycle 5
print("Wait")               # Print message
time.sleep(1)               # Wait for 1 second
duty = 2                    # Initial duty cycle
while duty <= 17:           # Loop until servo rotates 180 degrees
    servo.ChangeDutyCycle(duty)  # Change duty cycle to move servo
    time.sleep(0.5)         # Wait for servo to reach position
    duty = duty + 2         # Increase duty cycle
print("Turning back to 0 degrees")  # Print message
servo.ChangeDutyCycle(2)    # Move servo back to 0 degrees
time.sleep(1)               # Wait
servo.ChangeDutyCycle(0)    # Stop sending PWM signal
servo.stop()                # Stop PWM
g.cleanup()                 # Reset GPIO pins
print("Cleared")            # Program finished message
```

Output

```
Wait
Turning back to 0 degrees
Cleared
Servo motor turns 180 degrees
```

2. Interface Servo Motor with Multiple Cycles

```
import RPi.GPIO as g        # Import GPIO library
import time                 # Import time library
g.setmode(g.BOARD)          # Use board pin numbering
g.setwarnings(False)        # Disable warnings
g.setup(11, g.OUT)          # Set pin 11 as output
servo = g.PWM(11, 50)       # Create PWM object with 50Hz
servo.start(0)              # Start PWM with duty cycle 0
def set_angle(angle):       # Function to rotate servo to specific angle
    duty = 2 + (angle / 18) # Convert angle to duty cycle
    servo.ChangeDutyCycle(duty) # Send duty cycle to servo
    time.sleep(0.5)         # Wait for servo movement
    servo.ChangeDutyCycle(0)    # Stop signal
try:
    cycles = 3              # Number of cycles
    for i in range(cycles):     # Repeat for number of cycles
        print(f"Cycle {i+1}")
        for angle in range(0, 181, 30):  # Move servo from 0 to 180 in steps of 30
            print(f"Moving to {angle} degrees")
            set_angle(angle)
        for angle in range(180, -1, -30): # Move back from 180 to 0
            print(f"Moving to {angle} degrees")
            set_angle(angle)
    print("Completed all cycles")
except KeyboardInterrupt:       # If user stops program
    print("Process interrupted")
finally:
    servo.stop()            # Stop PWM
    g.cleanup()             # Reset GPIO pins
    print("GPIO cleaned up and program exited")
```

Output

```
Cycle 1
Moving to 0 degrees
Moving to 30 degrees
Moving to 60 degrees
Moving to 90 degrees
Moving to 120 degrees
Moving to 150 degrees
Moving to 180 degrees
Cycle 2...
Cycle 3...
Completed all cycles
Servo motor moves in step of 30 degrees
```

3. Interface Servo Motor with Angle as User Input

```
import RPi.GPIO as g        # Import GPIO library
from time import sleep      # Import sleep function
g.setmode(g.BOARD)          # Use board pin numbering
g.setwarnings(False)        # Disable warnings
control_pin = 11            # Servo control pin
g.setup(control_pin, g.OUT) # Set control pin as output
pwm = g.PWM(control_pin, 50) # Create PWM signal with 50Hz
set_angle = int(input("Enter servo motor rotation angle: "))  # Take angle from user
pwm.start(0)                # Start PWM
delay = 1                   # Delay time
duty_cycle = (set_angle / 18) + 2.5  # Convert angle to duty cycle
g.output(control_pin, True)  # Enable output
pwm.ChangeDutyCycle(duty_cycle) # Rotate servo to given angle
sleep(delay)                # Wait for servo movement
g.output(control_pin, False) # Disable output
pwm.ChangeDutyCycle(0)      # Stop PWM signal
pwm.stop()                  # Stop PWM
g.cleanup()                 # Reset GPIO pins
```

Output

```
Enter servo motor rotation angle: 75
Servo motor rotates to 75 degrees
```

Buzzer and Switch

```
import time                 # Import time library
import RPi.GPIO as gpio     # Import GPIO library
gpio.setwarnings(False)     # Disable warnings
gpio.setmode(gpio.BOARD)    # Use board numbering
buzzer = 38                 # Buzzer connected to pin 38
switch = 31                 # Switch connected to pin 31
gpio.setup(buzzer, gpio.OUT)  # Set buzzer as output
gpio.setup(switch, gpio.IN)   # Set switch as input
def sound_buzzer(event):    # Function triggered when switch pressed
    if event == switch:
        gpio.output(buzzer, True)  # Turn ON buzzer
        print("Buzzer ON")      # Display message
        time.sleep(1)           # Buzzer ON for 1 second
        gpio.output(buzzer, False)  # Turn OFF buzzer
        time.sleep(0.5)
gpio.add_event_detect(switch, gpio.RISING,  # Detect switch press
                      callback=sound_buzzer,
                      bouncetime=100)
try:
    while True:             # Run continuously
        time.sleep(1)
except KeyboardInterrupt:   # Stop program
    gpio.cleanup()          # Reset GPIO
```

Output

```
Buzzer on
```

PIR / IR Sensor with Buzzer

```
import RPi.GPIO as GPIO     # Import Raspberry Pi GPIO library
import time                 # Import time library for delays
sensor = 16
buzzer = 18                 # Buzzer connected to pin 18
GPIO.setmode(GPIO.BOARD)    # Use board pin numbering
GPIO.setup(sensor, GPIO.IN)   # Set sensor pin as input
GPIO.setup(buzzer, GPIO.OUT)  # Set buzzer pin as output
GPIO.output(buzzer, False)    # Initially turn OFF the buzzer
print("Initializing PIR Sensor...")  # Display initialization message
time.sleep(2)               # Wait for 2 seconds to stabilize sensor
print("PIR Ready...")       # Sensor ready message
print("")                   # Blank line for formatting
try:
    while True:             # Run program continuously
        if GPIO.input(sensor):      # Check if motion is detected
            GPIO.output(buzzer, True)  # Turn ON buzzer
            print("Motion Detected")  # Print message

            while GPIO.input(sensor):  # Wait while motion is present
                time.sleep(0.2)     # Small delay
        else:
            GPIO.output(buzzer, False) # Turn OFF buzzer if no motion
except KeyboardInterrupt:       # Stop program using CTRL + C
    GPIO.cleanup()              # Reset GPIO pins
    print("Program stopped and GPIO cleaned")
```

Output

```
Initializing PIR Sensor...
PIR Ready...
Motion Detected
Motion Detected
Motion Detected
```

LCD and DHT11 Temperature & Humidity Sensor

```
from RPLCD.gpio import CharLCD      # Import LCD control library
import Adafruit_DHT                 # Import DHT sensor library
import RPi.GPIO as GPIO             # Import Raspberry Pi GPIO library
import time                         # Import time library for delays
GPIO.setwarnings(False)             # Disable GPIO warnings
# Initialize LCD (20x4 LCD)
lcd = CharLCD(cols=20, rows=4,      # LCD size
              pin_rs=35,            # RS pin connected to GPIO 35
              pin_e=33,             # Enable pin connected to GPIO 33
              pins_data=[40, 38, 36, 32], # Data pins
              numbering_mode=GPIO.BOARD)  # Use board numbering
sensor = Adafruit_DHT.DHT11         # Define DHT11 sensor type
p = 17                              # GPIO pin where sensor is connected
print("Temp & Humidity in progress....")  # Display message
try:
    while True:                     # Run program continuously
        h, temp = Adafruit_DHT.read_retry(sensor, p)  # Read humidity & temperature
        print("Humidity =", str(h))  # Print humidity in terminal
        print("Temperature =", str(temp))  # Print temperature in terminal
        time.sleep(0.5)             # Small delay
        lcd.cursor_pos = (0, 0)     # Move cursor to first row
        lcd.write_string(f"Temperature:{temp}")  # Display temperature on LCD
        lcd.cursor_pos = (1, 0)     # Move cursor to second row
        lcd.write_string(f"Humidity:{h}")  # Display humidity on LCD
        time.sleep(1)               # Delay before next reading
except KeyboardInterrupt:           # Stop program using CTRL+C
    print("Program terminated")
    GPIO.cleanup()                  # Reset GPIO pins
```

Output (Terminal)

```
Temp & Humidity in progress....Humidity = 60
Temperature = 29 Humidity = 61 Temperature = 30
```

LCD Display Output

```
Temperature: 29
Humidity: 60
```

DHT11 Temperature & Humidity Sensor

```
import RPi.GPIO as GPIO         # Import Raspberry Pi GPIO library
import Adafruit_DHT             # Import DHT sensor library
import time                     # Import time library for delays
GPIO.setmode(GPIO.BOARD)        # Use board pin numbering
GPIO.setwarnings(False)         # Disable GPIO warnings
sensor = Adafruit_DHT.DHT11     # Define sensor type (DHT11)
pin = 17                        # Sensor connected to GPIO pin 17
print("Temperature & Humidity Monitoring")  # Display program start message
try:
    while True:                 # Run program continuously
        humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)
        # Read humidity and temperature from DHT11 sensor
        if humidity is not None and temperature is not None:
            # Check if sensor reading is successful
            print(f"Temp: {temperature:.1f}°C | Humidity: {humidity:.1f}%")
            # Display formatted temperature and humidity
        else:
            print("Sensor reading failed")  # Print error if reading fails
        time.sleep(2)           # Wait 2 seconds before next reading
except KeyboardInterrupt:       # Stop program using CTRL+C
    GPIO.cleanup()              # Reset GPIO pins
    print("Program stopped")
```

Output (Example)

```
Temperature & Humidity Monitoring
Temp: 29.0°C | Humidity: 60.0%
Temp: 30.0°C | Humidity: 62.0%
Temp: 29.5°C | Humidity: 61.0%
```

LED and Buzzer ON with Switch

```
import time                 # Import time library for delay
import RPi.GPIO as gpio     # Import Raspberry Pi GPIO library
gpio.setwarnings(False)     # Disable GPIO warnings
gpio.setmode(gpio.BOARD)    # Use board pin numbering
led = 40                    # LED connected to pin 40
switch = 31                 # Push button switch connected to pin 31
buzzer = 19                 # Buzzer connected to pin 19
gpio.setup(led, gpio.OUT, initial=0)  # Set LED pin as output and initially OFF
gpio.setup(buzzer, gpio.OUT)          # Set buzzer pin as output
gpio.setup(switch, gpio.IN)           # Set switch pin as input
def glow_led_and_sound_buzzer(event):  # Function triggered when switch is pressed
    if event == switch:             # Check if event is from switch
        gpio.output(led, True)      # Turn ON LED
        gpio.output(buzzer, True)   # Turn ON buzzer
        print("LED & Buzzer ON")    # Print message
        time.sleep(1.5)             # Wait for 1.5 seconds
        gpio.output(led, False)     # Turn OFF LED
        gpio.output(buzzer, False)  # Turn OFF buzzer
gpio.add_event_detect(switch, gpio.RISING,  # Detect switch press
                      callback=glow_led_and_sound_buzzer,
                      bouncetime=100)       # Debounce time
try:
    while True:                     # Run program continuously
        time.sleep(1)
except KeyboardInterrupt:           # Stop program using CTRL+C
    gpio.cleanup()                  # Reset GPIO pins
```

Output

```
LED & Buzzer ON
LED & Buzzer ON
```

Buzzer with Switch

1) Single LED using Raspberry Pi

```
import RPi.GPIO as GPIO  # Import GPIO library
import time              # Import time library for delay
GPIO.setmode(GPIO.BOARD)  # Use physical pin numbering
GPIO.setwarnings(False)  # Disable warnings
GPIO.setup(18, GPIO.OUT)  # Set pin 18 as output for LED
while True:              # Run continuously
    print("LED ON")
    GPIO.output(18, GPIO.HIGH)  # Turn LED ON
    time.sleep(1)
    print("LED OFF")
    GPIO.output(18, GPIO.LOW)  # Turn LED OFF
    time.sleep(1)
```

Output

```
LED blinks continuously (ON for 1 second and OFF for 1 second).
```

2) Multiple LEDs using Raspberry Pi

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)
GPIO.setup(16, GPIO.OUT)
GPIO.setup(18, GPIO.OUT)
GPIO.setup(22, GPIO.OUT)
while True:
    print("LEDs ON")
    GPIO.output(16, GPIO.HIGH)
    GPIO.output(18, GPIO.HIGH)
    GPIO.output(22, GPIO.HIGH)
    time.sleep(1)
    print("LEDs OFF")
    GPIO.output(16, GPIO.LOW)
    GPIO.output(18, GPIO.LOW)
    GPIO.output(22, GPIO.LOW)
    time.sleep(1)
```

Output

```
All LEDs turn ON and OFF simultaneously.
```

3) Single LED with Switch

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)
GPIO.setup(18, GPIO.OUT) # LED pin
GPIO.setup(16, GPIO.IN,
pull_up_down=GPIO.PUD_DOWN) # Switch pin
while True:
if GPIO.input(16) == GPIO.HIGH: # If switch pressed
print("Switch ON - LED ON")
GPIO.output(18, GPIO.HIGH)
else:
print("Switch OFF - LED OFF")
GPIO.output(18, GPIO.LOW)
time.sleep(0.5)
```

Output

```
When the switch is pressed → LED turns ON
When the switch is released → LED turns OFF
```